

Perceptron Multicapa – Ejercicio 01

1. Enlaces de Colab (o archivos .ipynb) de las dos notebooks modificadas, con las corridas originales y las profundas.

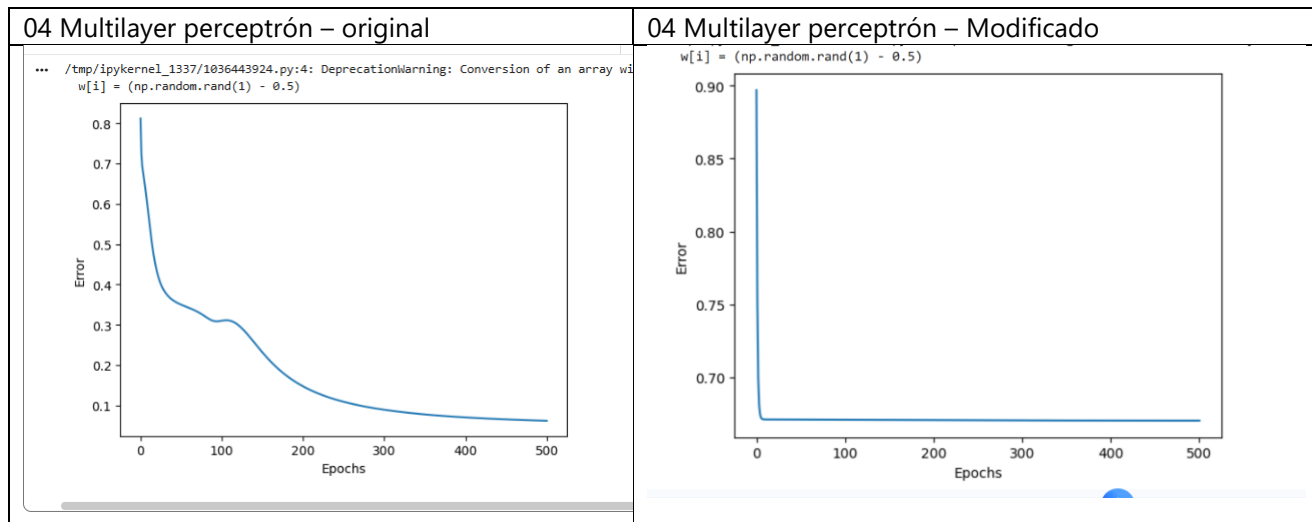
04 Multilayer perceptrón:

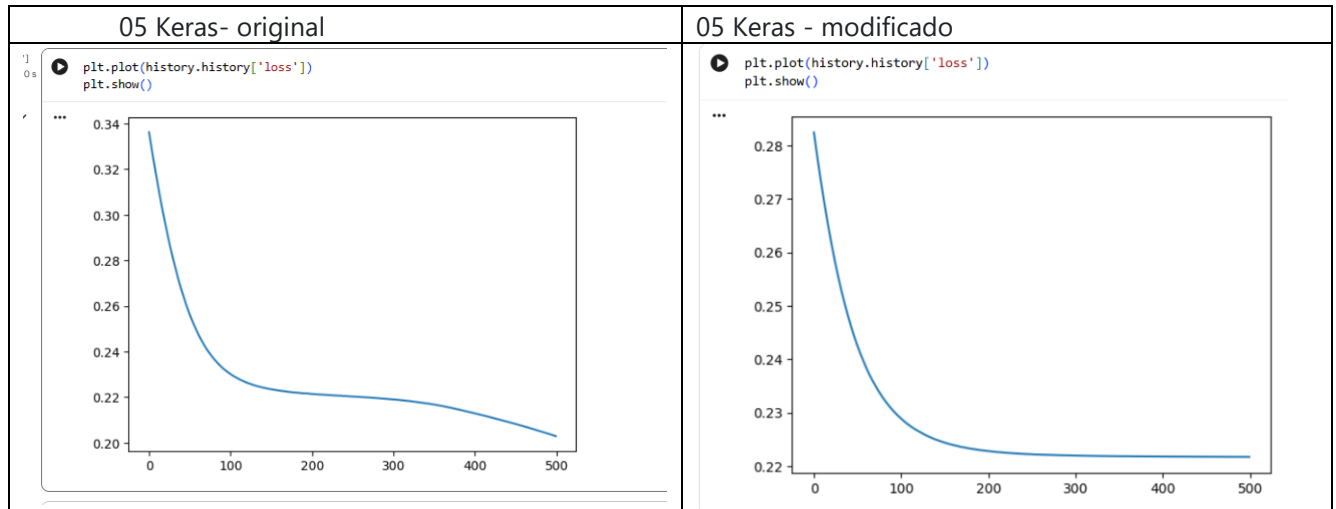
https://drive.google.com/file/d/136u8slwF8QpcCBLEpXj6_2mPp6h317q7/view?usp=sharing

05 Keras - multilayer perceptron – iris:

https://drive.google.com/file/d/1QoWvO0DapCCzxBrlw_vF-_doA757kbmH/view?usp=sharing

2. Capturas: curvas de error/pérdida de las cuatro corridas y los dos model.summary() de Keras.





Model Summary

Original

```
model.summary()
```

Model: "sequential"

Layer (type)	Output Shape	Param #
layer1 (Dense)	(None, 3)	15
layer2 (Dense)	(None, 3)	12

Total params: 27 (108.00 B)
Trainable params: 27 (108.00 B)
Non-trainable params: 0 (0.00 B)

Model summary

Editada

```
model.summary()
```

...

Model: "sequential"

Layer (type)	Output Shape	Param #
layer1 (Dense)	(None, 3)	15
layer2 (Dense)	(None, 3)	12
layer3 (Dense)	(None, 3)	12
layer4 (Dense)	(None, 3)	12

Total params: 51 (204.00 B)
Trainable params: 51 (204.00 B)
Non-trainable params: 0 (0.00 B)

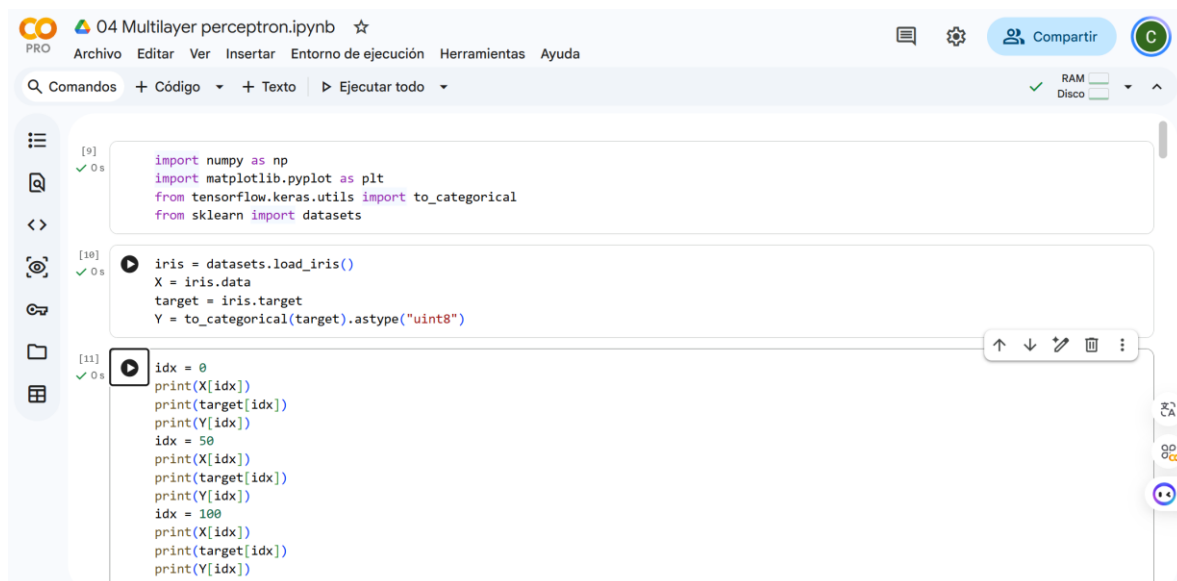
3. Un breve reporte (media página a una página) que responda:

Añadir mas capas en Numpy estancó el error cerca del 0.7 para todas las épocas cuando con solo 2 capas, el error al final de las épocas fue de .07, con keras al añadir capas el error inicializó más bajo y se fue acercando a 0 en menos épocas pero se estancó cerca de .22 desde la época 200 hasta la 500 cuando con solo dos capas el error al final de las épocas fue menor a .22, pero considerando que son dos escalas diferentes el comportamiento fue igual al agregar más capas con numpy keras se estancó el error cerca de .22.

Las curvas se parecen con la red profunda y con solo 2 capas difieren probablemente por la cantidad de veces que se corrigen los pesos, NumPy corrige después de cada flor, o sea 150 correcciones por vuelta; Keras junta 32 flores y corrige una sola vez, unas 5 correcciones por vuelta.

En iris tiene sentido que una red más profunda no aprenda mejor, cuando la red se equivoca, la corrección nace en la última capa y tiene que pasarse hacia atrás de capa en capa. Y si esa primera capa nunca aprende a leer bien los datos, las demás no tienen con qué trabajar y lo podemos notar en el número donde se estanca .22 es lo que da el error cuando la red dejó de distinguir especies. En NumPy el estancamiento ocurre antes de la época 10 y la curva queda como una recta las 490 restantes en Keras se aplanan alrededor de la época 200.

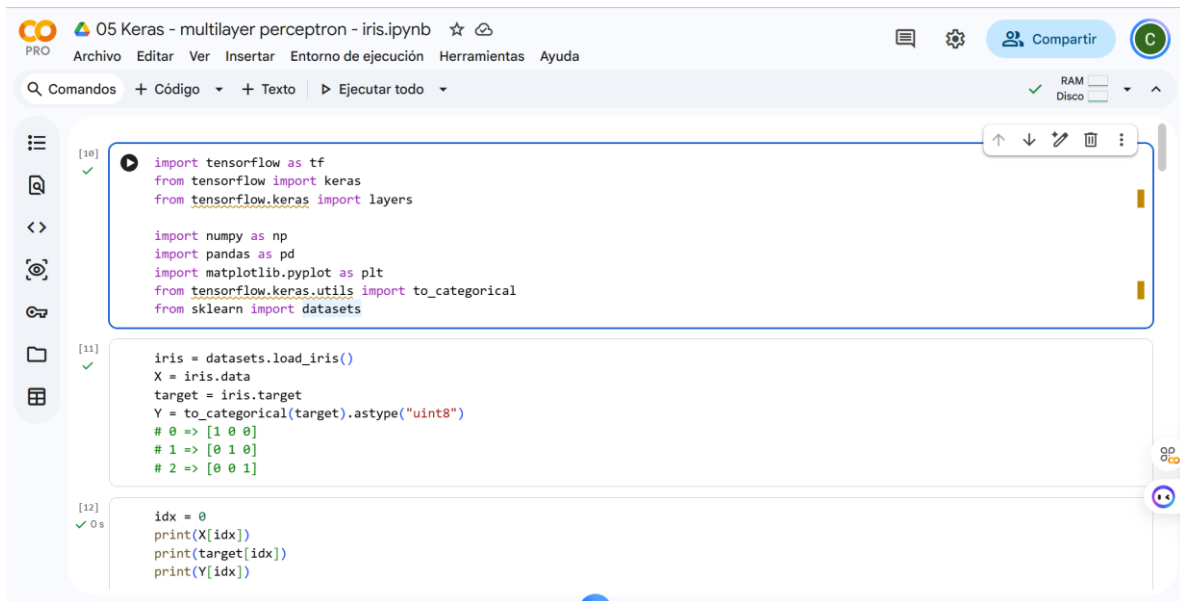
4. Evidencias de haber ejecutado en Colab (captura del entorno Colab o del menú Runtime).



```
[9] import numpy as np
import matplotlib.pyplot as plt
from tensorflow.keras.utils import to_categorical
from sklearn import datasets

[10] iris = datasets.load_iris()
X = iris.data
target = iris.target
Y = to_categorical(target).astype("uint8")

[11] idx = 0
print(X[idx])
print(target[idx])
print(Y[idx])
idx = 50
print(X[idx])
print(target[idx])
print(Y[idx])
idx = 100
print(X[idx])
print(target[idx])
print(Y[idx])
```



The screenshot displays a Jupyter Notebook titled "05 Keras - multilayer perceptron - iris.ipynb". The interface includes a top menu bar with options like "Archivo", "Editar", "Ver", "Insertar", "Entorno de ejecución", "Herramientas", and "Ayuda". A left sidebar contains icons for file management and execution. The main area shows three code cells, each with a green checkmark indicating successful execution.

```
[10] import tensorflow as tf
      from tensorflow import keras
      from tensorflow.keras import layers

      import numpy as np
      import pandas as pd
      import matplotlib.pyplot as plt
      from tensorflow.keras.utils import to_categorical
      from sklearn import datasets

[11] iris = datasets.load_iris()
      X = iris.data
      target = iris.target
      Y = to_categorical(target).astype("uint8")
      # 0 => [1 0 0]
      # 1 => [0 1 0]
      # 2 => [0 0 1]

[12] idx = 0
      print(X[idx])
      print(target[idx])
      print(Y[idx])
```